



UMĚLÁ INTELIGENCE

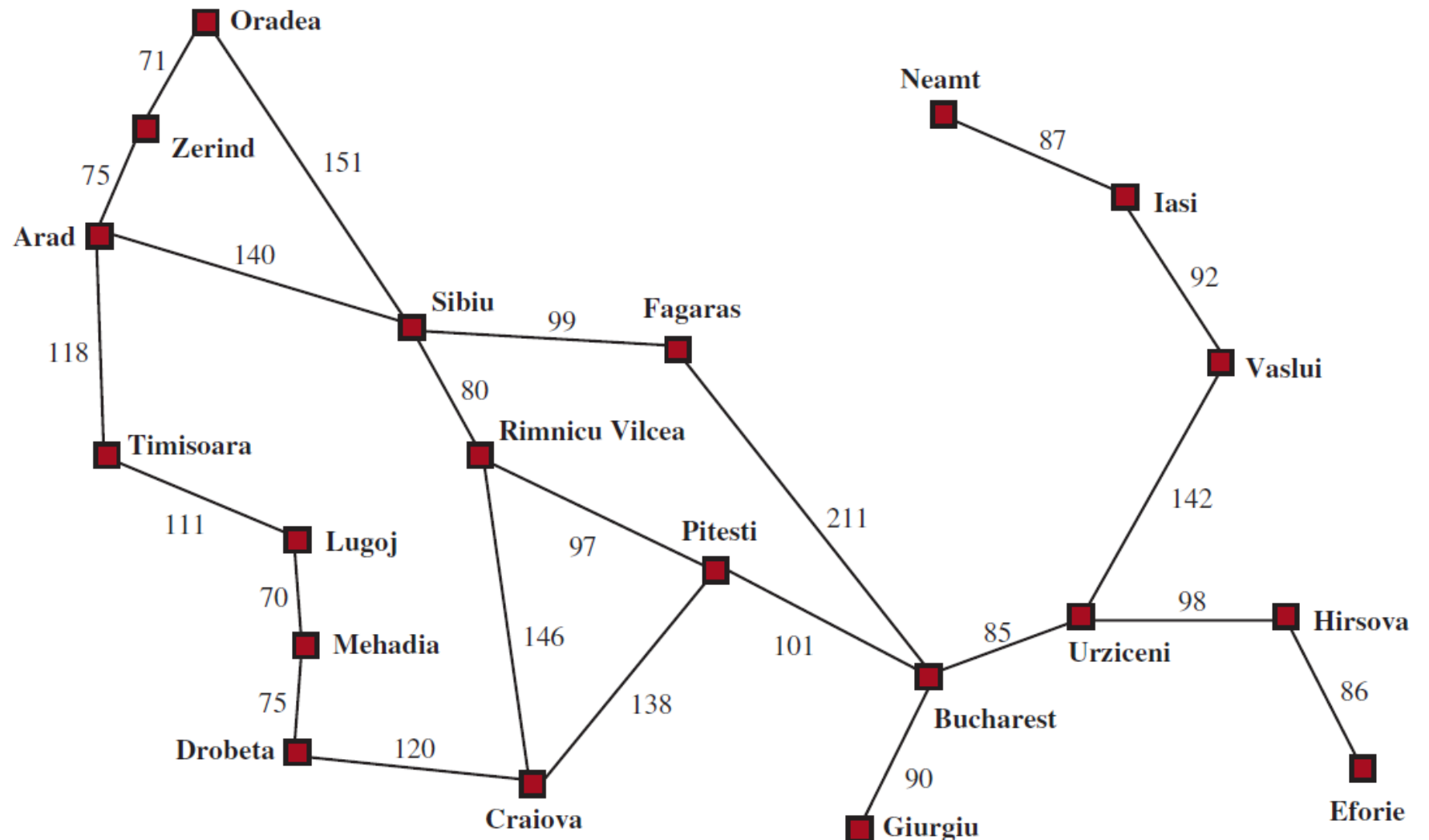
Přednáška 3

Řešení problémů prohledáním

AGENT ŘEŠÍCÍ PROBLÉMY

- prostředí: epizodické, plně pozorovatelné, deterministické, statické, diskrétní, známé, 1 agent
 - správná akce není na první pohled zřejmá
 - agent se „dívá“ dopředu (plánuje), aby našel posloupnost akcí vedoucích k cíli
 - příslušný výpočetní proces = prohledávání:
 - informované – agent má odhad vzdálenosti od cíle
 - neinformované
 - atomická reprezentace agenta
-

PŘÍKLAD – AGENT MÁ CÍL DOSTAT SE Z ARADU DO BUKUREŠTI



PŘÍKLAD – AGENT MÁ CÍL DOSTAT SE Z ARADU DO BUKUREŠTI

- předpoklad - agent má přístup k informacím o světě (mapa)
 - 4-fázový proces řešení problému:
 - **formulace cíle** - agent přijme cíl (dojet do Bukurešti)
 - **formulace problému** - agent navrhne popis stavů a akcí nutných k dosažení cíle— abstraktní model příslušné části světa (akce = cesta z aktuálního do sousedního města, stav = aktuální město)
 - **prohledávání** – před vlastní akcí ve skutečném světě, agent simuluje akce na svém modelu, prohledává stavový prostor, dokud nenajde posloupnost akcí, která vede k cíli nebo zjistí, že řešení neexistuje; výsledná sekvence = řešení.
 - **realizace** - agent provede nalezenou posloupnost akcí
-

PŘÍKLAD – AGENT MÁ CÍL DOSTAT SE Z ARADU DO BUKUREŠTI

- v plně pozorovatelném, deterministickém a známém prostředí je řešením **jednoznačná posloupnost akcí** Arad – Sibiu – Fagaras – Bukurešť
 - pokud má agent správný model a nalezne řešení, pak může během vykonávání akcí ignorovat pozorování (systém bez zpětné vazby)
 - pro nepřesný model nebo stochastické prostředí je třeba pozorovat stavy a na základě změn případně upravit plánované akce (systém se zpětnou vazbou)
-

DEFINICE ÚLOHY PROHLEDÁVÁNÍ

- množina možných stavů prostředí = stavový prostor
 - **počáteční stav** – zde agent začíná prohledávání
 - **množina cílových stavů** (jeden nebo více)
 - **dostupné akce** – množina akcí, které lze provést v daném stavu
 - **přechodový model** - popisuje, co každá akce způsobí: stav s + akce $a \rightarrow$ nový stav s'
 - **účelová funkce stavu** $c(s, a, s')$ – numerická hodnota – cena za uskutečnění akce a ve stavu s pro dosažení stavu s' – agent řešící problémy používá účelovou funkci, která odráží jeho měřítko výkonnosti (např. pro agenta hledajícího cestu je to vzdálenost v km, případně čas potřebný k dokončení celé akce)
-

ÚLOHA PROHLEDÁVÁNÍ – ZÁKLADNÍ POJMY

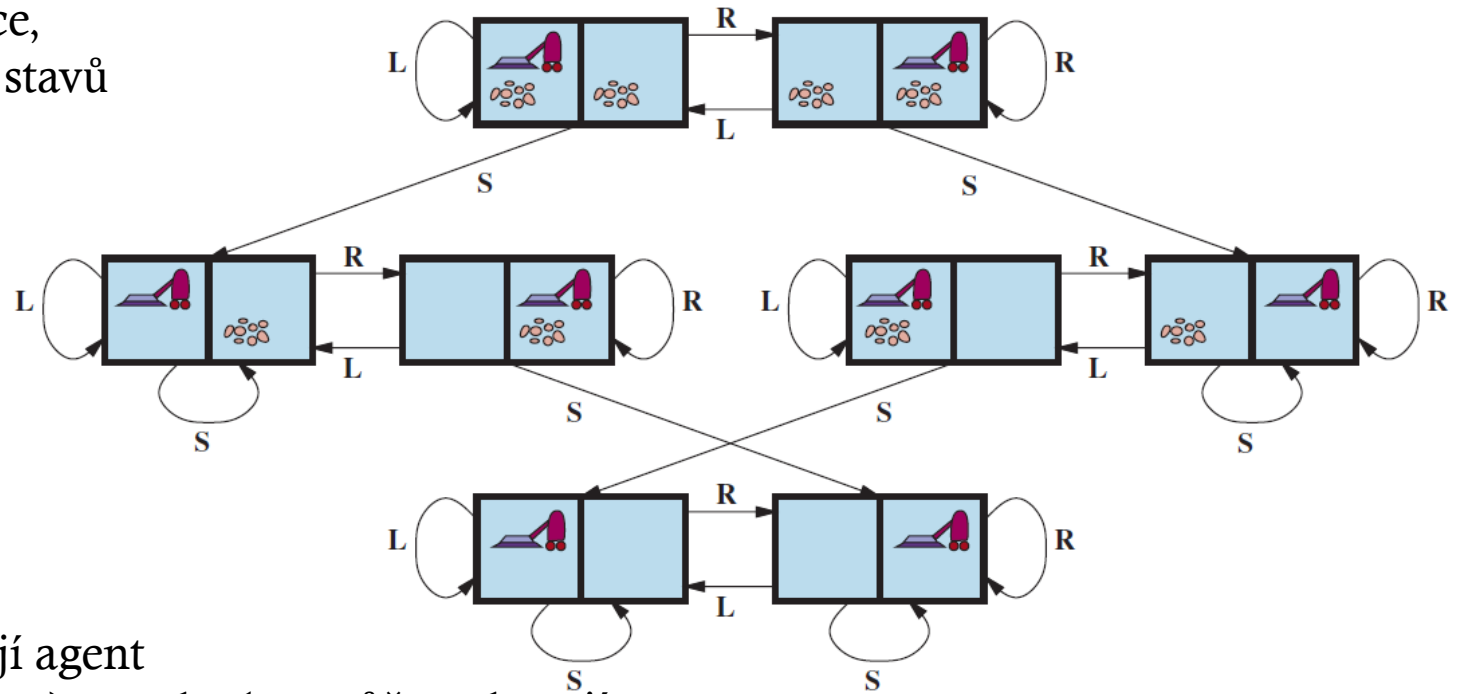
- posloupnost akcí = cesta
 - řešení = cesta z počátečního do cílového stavu
 - celková cena cesty je součtem cen jednotlivých akcí
 - jednotlivé ceny jsou kladná čísla
 - optimální řešení má ze všech možných řešení nejnižší cenu
 - stavový prostor je reprezentovaný **orientovaným grafem**
 - uzly = stavy
 - hrany = akce
 - formulace problému = abstraktní model → zjednodušení reality
-

PŘÍKLADY PROBLÉMŮ PRO AGENTA ŘEŠÍCÍHO PROBLÉMY

- standardizované problémy
 - simulace, ilustrace a procvičení různých metod řešení, benchmarky
 - „mřížkový svět“ (grid world) – agent se pohybuje v rámci buněk 2D čtvercového pole, buňky obsahují objekty, se kterými agent může manipulovat nebo překážky zabráňující vstupu do buňky
 - reálné problémy
 - navigace robota
 - hledání cesty
 - problém obchodního cestujícího
 - návrh VLSI obvodů
 - automatizované kompletování výrobků
 - ...
-

MŘÍŽKOVÝ SVĚT - SVĚT VYSAVAČE

- stavy → jaké objekty jsou v každé buňce, obecně prostředí s n buňkami má $n \cdot 2^n$ stavů
- počáteční stav – libovolný
- akce:
 - L = pohyb vlevo,
 - R = pohyb vpravo
 - S = vysávání,ve více-buňkovém prostředí přidat
 - U = nahoru
 - D = dolů
- přechodový model:
 - S odstraní smetí z buňky, na které stojí agent
 - L (R) - agent se pohybuje vlevo (vpravo), a pokud nemůže, tak stojí
- cílový stav – všechny buňky jsou čisté
- cena akce – každá akce stojí 1



MŘÍŽKOVÝ SVĚT - HRA 9 (15)

- stavy → poloha všech dílků
- počáteční stav – libovolný
- akce - pohyb prázdné buňky vlevo, vpravo, nahoru, dolů
- přechodový model – zobrazení aktuálního stavu a akce na nový stav; např. aplikace akce vlevo na výchozí stav prohodí 5 a mezeru
- cílový stav – požadované seřazení čísel
- cena akce – každá akce stojí 1

7	2	4
5		6
8	3	1

výchozí stav

	1	2
3	4	5
6	7	8

cílový stav

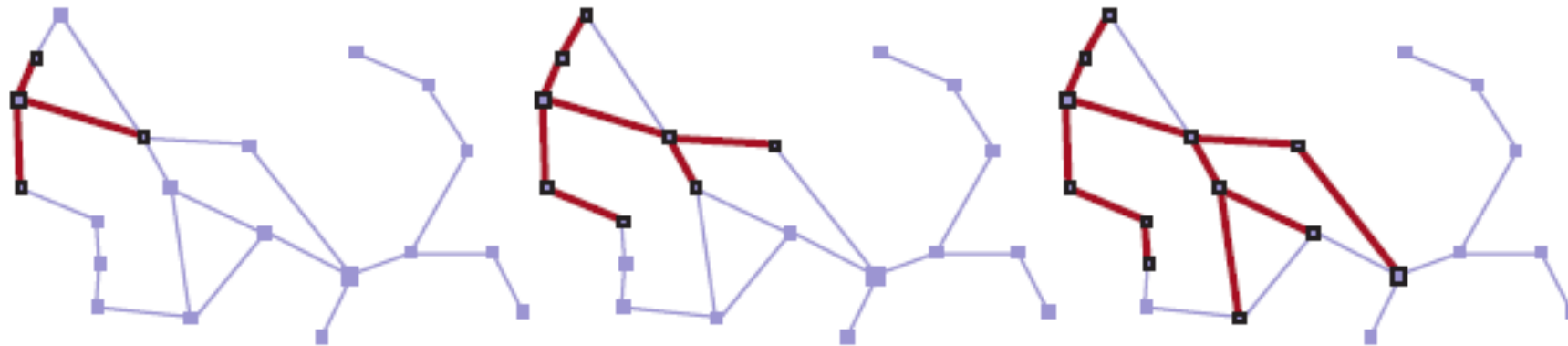
HLEDÁNÍ CESTY

Příklad reálného problému - cestování letadlem - řešení pomocí webového plánovače:

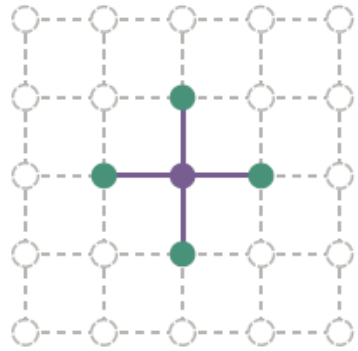
- **Stavy:** Každý stav obsahuje lokaci (např. letiště) a aktuální čas. Dále stav musí obsahovat extra informaci o “historických” aspektech, neboť cena akce (letový segment) může záviset na předchozích segmentech, letových příplatcích, statusu (domácí nebo mezinárodní let).
 - **Počáteční stav:** Výchozí letiště.
 - **Akce:** Letět z aktuální lokace, v libovolné třídě, s odletem pozdějším, než je aktuální čas, s dostatkem času pro případný přesun v rámci letiště.
 - **Přechodový model:** Výsledný stav po absolvování letu, cílové letiště je novou lokací a čas příletu je novým časem.
 - **Cílový stav:** Cílová destinace; někdy může být cíl komplexnější - např. "Dolet do cíle bez přestupu"
 - **Cena akce:** Kombinace ceny v penězích, doby čekání na přestup, doba letu, kvalita sedadel, denní doba, typ letadla, atd.
-

PROHLEDÁVACÍ ALGORITMY

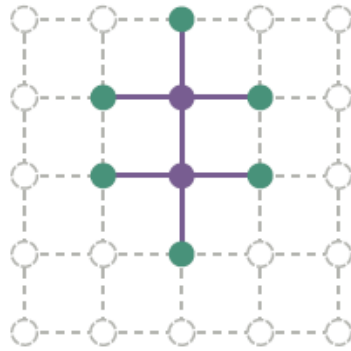
- vstupem je řešený problém (prohledávání), výstupem je řešení nebo informace o neúspěchu
- hledání cesty z počátečního do cílového stavu, ve stavovém prostoru vytváříme prohledávací strom
- kořen stromu = počáteční stav, uzly – rodič (předchůdce) + potomek (následník)
- základ prohledávání - výběr, který uzel z množiny potomků zpracovat nejdříve, zbytek odložit na pozdější zpracování



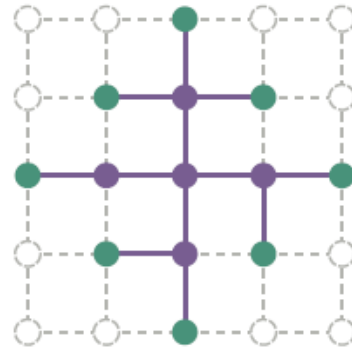
PROHLEDÁVACÍ ALGORITMY - SEPARAČNÍ VLASTNOST



(a)



(b)



(c)

Hranice (zelená) odděluje vnitřek (fialová) od vnějšku (slabě čárkovaná).

Hranice je množina uzlů (a odpovídajících stavů), které byly dosaženy, ale ještě ne expandovány.

Vnitřek je množina uzlů (a odpovídajících stavů), které byly expandovány.

Vnějšek je množina stavů, které ještě nebyly dosaženy.

Na obr. (a) je expandován pouze kořen. Na obr. (b) je expandován horní hraniční uzel.

Na obr. (c) jsou expandováni zbývající následníci kořene ve směru hodinových ručiček.

PROHLEDÁVACÍ ALGORITMY

Měřítko výkonnosti:

- **Úplnost** - algoritmus najde řešení, pokud existuje, anebo hlásí neúspěch
 - **Cenová optimalita** - nalezené řešení má nejmenší cenu ze všech možných řešení
 - **Časová náročnost** - čas potřebný k nalezení řešení (počet potřebných stavů a akcí)
 - **Prostorová náročnost** - velikost potřebné paměti
-

USPOŘÁDANÉ VYHLEDÁVÁNÍ - "BEST-FIRST SEARCH"

- obecný postup pro výběr „nejlepšího“ uzlu
- každý uzel je ohodnocen pomocí hodnotící funkce $f(n)$
- v každé iteraci vybíráme z fronty uzel n s minimální hodnotou $f(n)$, zkoumáme, je-li cílový, a pokud ne, generujeme potomky
- potomek je přidán do fronty, pokud je nově objeven anebo je objeven znovu, a to cestou, která má nižší náklady než jakákoli předchozí cesta
- algoritmus vrátí buď cestu k cíli nebo informaci o neúspěchu.

Použitím různých funkcí $f(n)$ dostaneme různé prohledávací algoritmy.

USPOŘÁDANÉ VYHLEDÁVÁNÍ - "BEST-FIRST SEARCH"

Algoritmus prochází graf od počátečního uzlu k cílovému uzlu tak, že v každém kroku vybírá z hraničních uzlů ten, který má minimální hodnotu hodnotící funkce.

Hodnotící funkce **$f(n)$** pro uzel **n** určuje, jak "dobrý" je tento uzel.

Hodnotící funkce může být například Euklidovská vzdálenost, pokud uzly reprezentují body v prostoru.

Základní vlastnosti:

- Optimalita: algoritmus je optimální, pokud je hodnotící funkce neklesající.
 - Úplnost: existuje-li řešení, algoritmus ho najde.
 - Časová složitost: Časová složitost je závislá na počtu uzlů, které musí být prozkoumány, což je v nejhorším případě exponenciální v závislosti na hloubce cílového uzlu.
 - Prostorová složitost: Prostorová složitost je také exponenciální, v paměti jsou uchovávány všechny uzly.
-

STRUKTURA PROHLEDÁVACÍHO ALGORITMU

Uzel je struktura se 4 komponentami:

- stav - stav, kterému odpovídá tento uzel
- rodič - předchozí uzel
- akce - akce uskutečněná v předchozím stavu, která generovala tento uzel
- cena cesty - **$g(n)$** - celková cena cesty z počátečního stavu do uzlu **n**

Uzly k expanzi jsou ve frontě/zásobníku

- prioritní fronta - přednost mají uzly s nejmenší hodnotou **$f(n)$**
 - FIFO - uzly se zpracovávají v pořadí, v jakém jsou dosaženy
 - LIFO - nejprve se zpracují naposledy přidané uzly
-

STROMOVÉ VS. GRAFOVÉ PROHLEDÁVÁNÍ

Pokud se v prohledávacím stromu vrátíme znovu do některého z předchozích stavů, jedná se o cyklus. Cyklus je speciálním případem redundantní cesty.

Redundantní cesta je taková, která je "dražší" než jiná, již nalezená.

Eliminací redundantních cest zrychlíme prohledávání. K tomu je třeba si pamatovat všechny prošlé stavy, což ale zase zvyšuje nároky na paměť.

Varianty prohledávání:

- **grafové prohledávání** - algoritmus si pamatuje všechny dříve dosažené stavy (viz „best-first search“), což umožní odhalit všechny nadbytečné cesty a ponechat si pouze tu nejlepší cestu ke každému stavu
 - **stromové prohledávání** - prošlé stavy se neukládají, ušetří se tak místo v paměti, ale algoritmus je pomalejší
 - **kompromis** - algoritmus hledá cykly, ale ne redundantní cesty obecně - využívá řetězec ukazatelů na rodiče pro zjištění, zda se aktuální stav už neobjevil na cestě dříve.
-

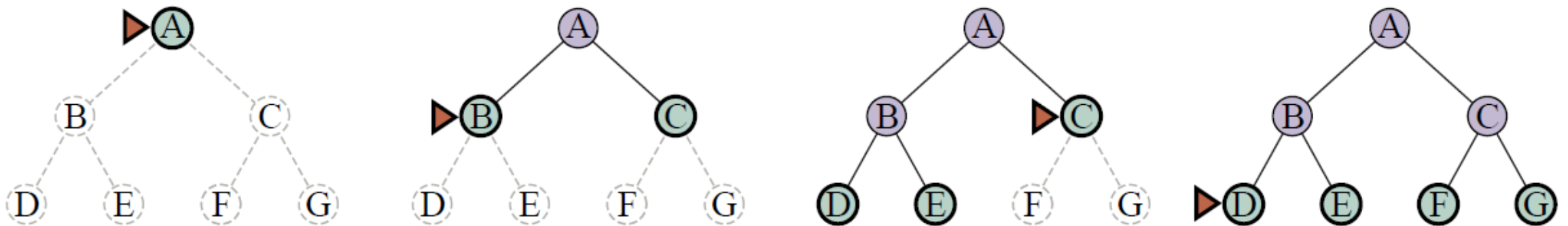
NEINFORMOVANÉ PROHLEDÁVÁNÍ

- hodnotící funkce je rovna ceně cesty od počátečního do aktuálního stavu v uzlu **n** , tj. **$f(n) = g(n)$**
 - algoritmy - prohledávání:
 - do šířky – BFS
 - Dijkstrův algoritmus
 - do hloubky – DFS
 - DFS s omezenou hloubkou
 - obousměrné prohledávání
-

NEINFORMOVANÉ PROHLEDÁVÁNÍ

Prohledávání do šířky (BFS)

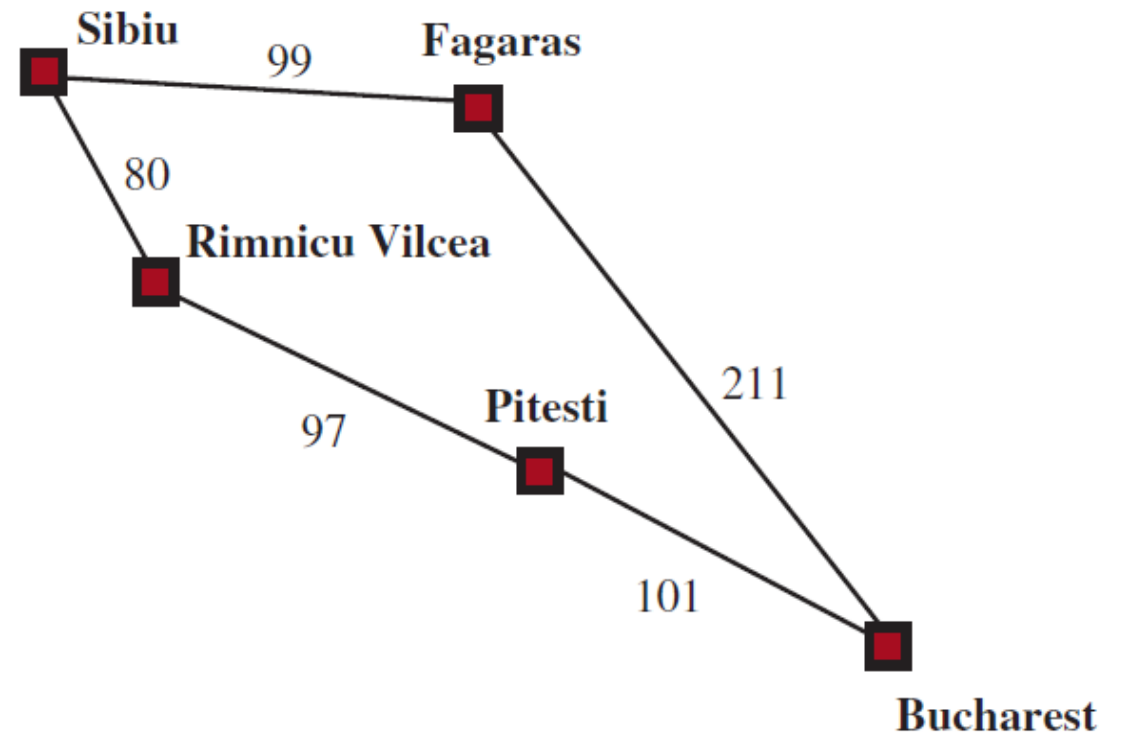
- $f(n)$ -- hloubka uzlu (počet akcí k dosažení cíle)
- FIFO fronta, všechny uzly zůstávají v paměti
- kompletní algoritmus, vždy najde řešení
- cenově optimální, pokud všechny akce mají stejnou hodnotu
- časová a prostorová náročnost $O(b^d)$, b = počet potomků uzlu, d = hloubka



NEINFORMOVANÉ PROHLEDÁVÁNÍ

Prohledávání „uniform-cost“ neboli Dijkstrův algoritmus

- akce s různými cenami
- prioritní fronta
- $f(n)$ = cena cesty od kořene k aktuálnímu uzlu
- kompletní
- cenově optimální
- časově i prostorově náročnější než BFS



NEINFORMOVANÉ PROHLEDÁVÁNÍ

Dijkstrův algoritmus

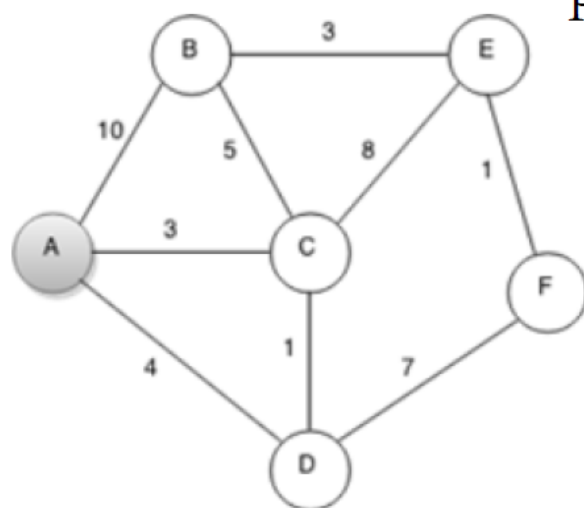
- hledání nejkratších cest z kořene s (počátečního vrcholu) do ostatních vrcholů grafu
 - pro grafy s nezáporným ohodnocením hran
 - na začátku přiřadíme uzlům určité hodnoty a postupně je budeme vylepšovat
 - rozšíření procházení grafu do šířky, přičemž místo datové struktury fronta zde využíváme prioritní frontu (fronta, ve které mohou předbíhat uzly s nejkratší aktuální vzdáleností od zdroje)
 - algoritmus tedy systematicky postupuje od výchozího uzlu k cílové destinaci a aktualizuje vzdálenosti. Výstupem algoritmu je pak délka nejkratší cesty z výchozího uzlu do všech ostatních uzlů
-

DIJKSTRŮV ALGORITMUS - PSEUDOKÓD

1. každému uzlu přiřad' aktuální vzdálenost od počátečního uzlu, počáteční uzel má vzdálenost 0, ostatní uzly nekonečno
 2. označ výchozí uzel jako aktuální uzel a všechny ostatní uzly jako nenavštívené
 3. podívej se na všechny nenavštívené sousedy aktuálního uzlu a vypočítej jejich vzdálenost od počátečního uzlu - pokud je aktuální vzdálenost menší než dříve zaznamenaná vzdálenost tohoto souseda, pak ji přepiš novou menší hodnotou
Pozn.: aktuální uzel odebereme a označíme jako navštívený, sousední uzly aktuálního uzlu zůstávají v množině nenavštívených
 4. pokud byl cílový uzel označen jako navštívený nebo nejmenší aktuální vzdálenost mezi uzly v množině nenavštívených uzlů je nekonečno, ukonči algoritmus
 5. z množiny nenavštívených uzlů vyber uzel, který má nejmenší aktuální vzdálenost od výchozího uzlu, nastav jej jako „aktuální“ uzel a vrať se k bodu 3
-

NEINFORMOVANÉ PROHLEDÁVÁNÍ

Dijkstrův algoritmus - Příklad: Vyhledejte nejkratší cestu z počátečního uzlu A do všech ostatních uzlů ohodnoceného grafu.



Postup:

A	B	C	D	E	F	množina X
0	10, A	3, A	4, A	max	max	A
-	8, C	-	4, A	11, C	max	A, C
-	8, C	-	-	11, C	11, D	A, C, D
-	-	-	-	11, C	11, D	A, C, D, B
-	-	-	-	-	11, D	A, C, D, B, E
-	-	-	-	-	-	A, C, D, B, E

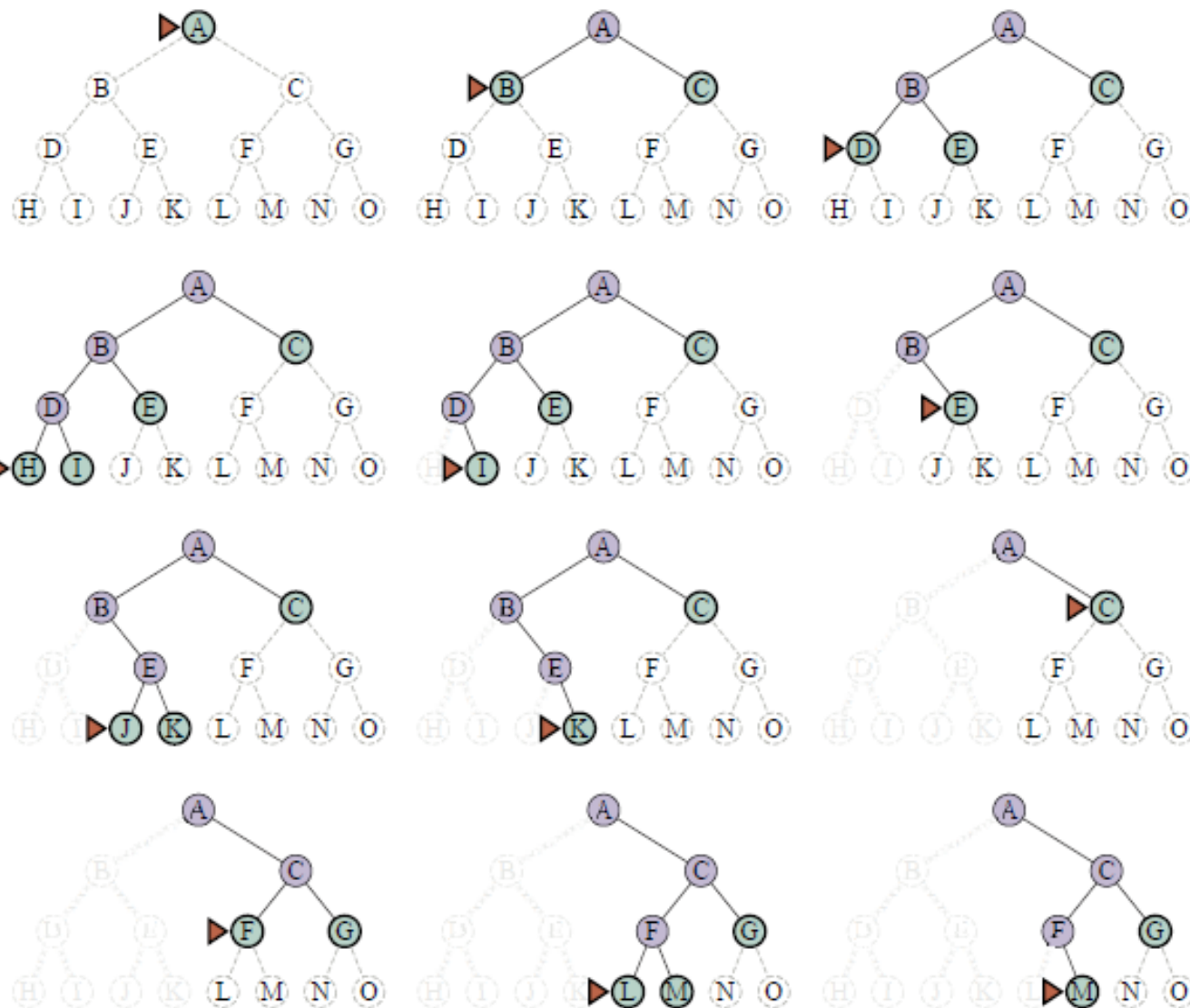
Výsledek:

A	B	C	D	E	F
0	8, C	3, A	4, A	11, C	11, D

NEINFORMOVANÉ PROHLEDÁVÁNÍ

Prohledávání do hloubky (DFS)

- $f(n)$ = záporná hodnota hloubky
- stromový algoritmus
- není cenově optimální
- kompletní pro konečné stavové prostory
- paměťová náročnost $O(bm)$,
b = počet potomků, m = hloubka



NEINFORMOVANÉ PROHLEDÁVÁNÍ

Varianta DFS - Prohledávání s omezenou hloubkou

- pevně zadaný limit nebo iterativně se zvyšující
- časová náročnost $O(b^l)$, paměťová náročnost $O(bl)$

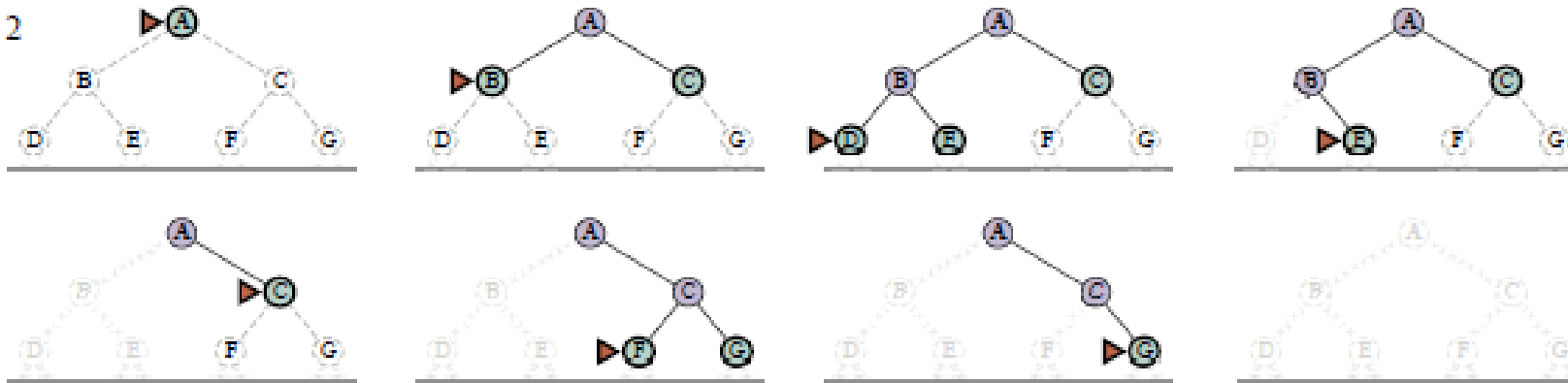
limit: 0



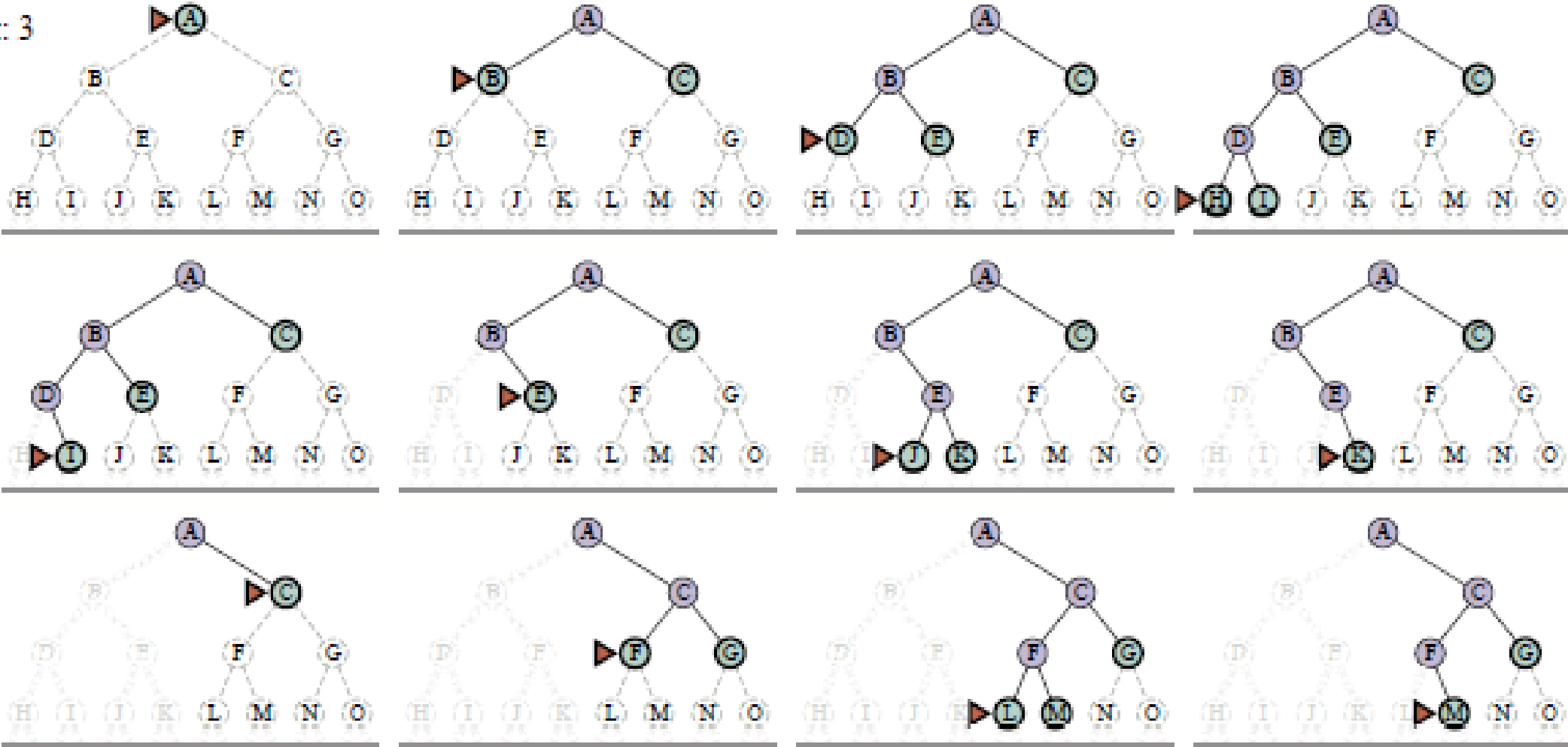
limit: 1



limit: 2



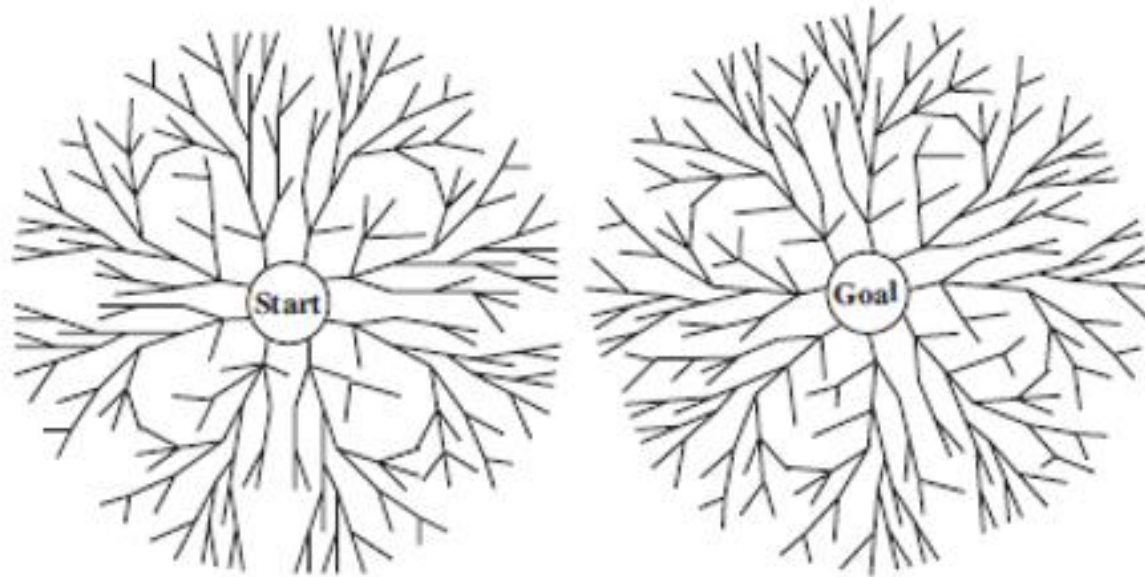
limit: 3



NEINFORMOVANÉ PROHLEDÁVÁNÍ

Obousměrné prohledávání

- současné prohledávání z počátečního stavu (vpřed) a z cílového stavu (zpět)
- ukládají se dvě fronty a dvě tabulky dosažených stavů
- spojí-li se fronty, získáme řešení (nemusí být nejlepší)



NEINFORMOVANÉ PROHLEDÁVÁNÍ

Porovnání algoritmů

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Figure 3.15 Evaluation of search algorithms. b is the branching factor; m is the maximum depth of the search tree; d is the depth of the shallowest solution, or is m when there is no solution; ℓ is the depth limit. Superscript caveats are as follows: ¹ complete if b is finite, and the state space either has a solution or is finite. ² complete if all action costs are $\geq \epsilon > 0$; ³ cost-optimal if action costs are all identical; ⁴ if both directions are breadth-first or uniform-cost.

INFORMOVANÉ PROHLEDÁVÁNÍ

- využívá odhad o vzdálenosti do cíle pomocí heuristické funkce $h(n)$
- $h(n)$ = odhadovaná cena nejlevnější cesty z daného stavu do cíle
- příklad $h(n)$: letecká vzdálenost mezi dvěma městy

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD} —straight-line distances to Bucharest.

INFORMOVANÉ PROHLEDÁVÁNÍ

Hladový algoritmus (greedy best-first search)

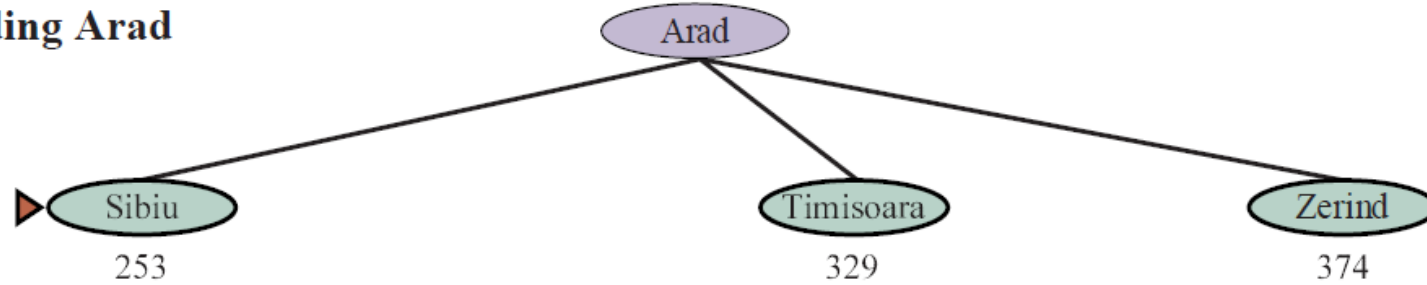
- evaluační funkce je rovna přímo heuristice, tj. $f(n) = h(n)$
- zpracuje přednostně uzel s nejnižší hodnotou $h(n)$
- rychlé dosažení cíle (na nejmenší počet kroků)
- nalezené řešení nemusí být optimální (tj. nejlevnější)

INFORMOVANÉ PROHLEDÁVÁNÍ - HLADOVÝ ALGORITMUS

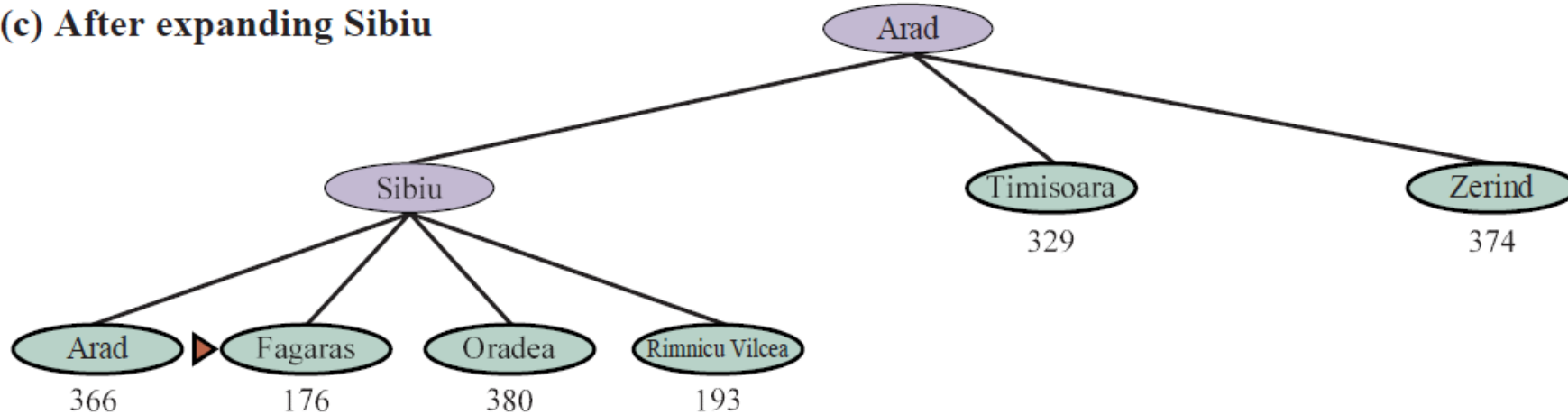
(a) The initial state



(b) After expanding Arad

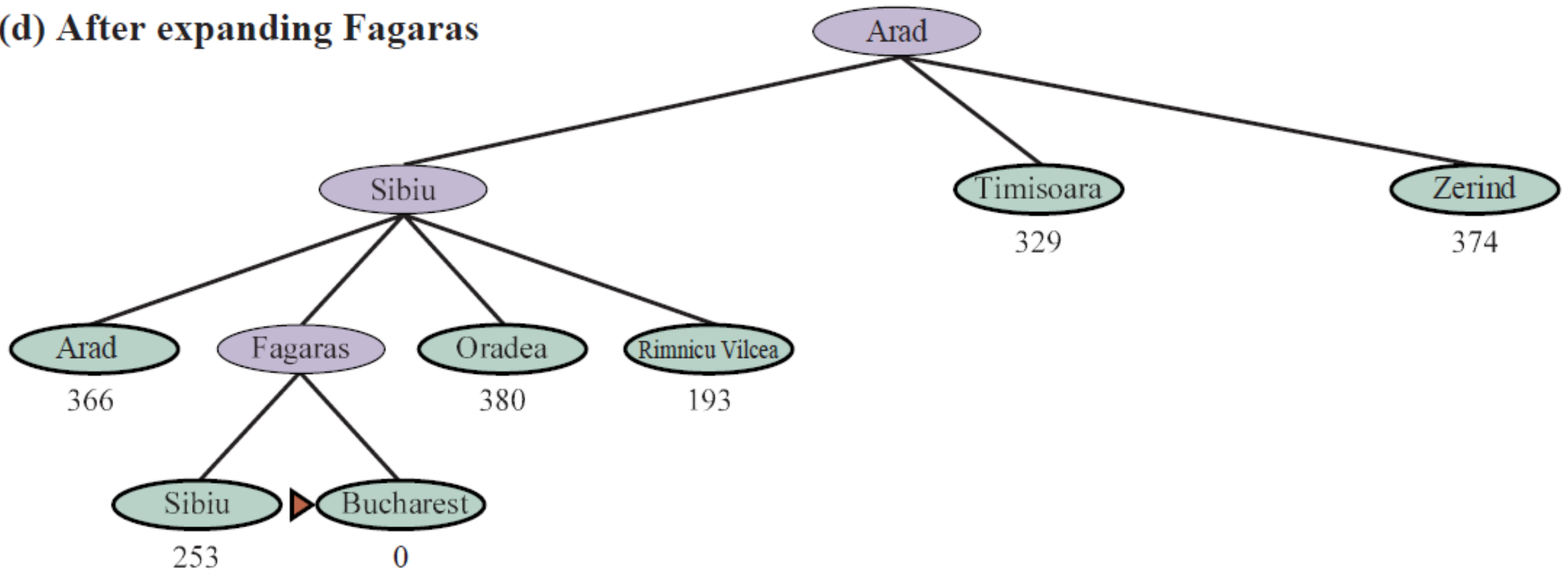


(c) After expanding Sibiu



INFORMOVANÉ PROHLEDÁVÁNÍ - HLADOVÝ ALGORITMUS

(d) After expanding Fagaras



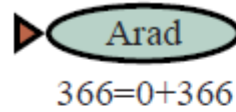
INFORMOVANÉ PROHLEDÁVÁNÍ

A* - A star prohledávání

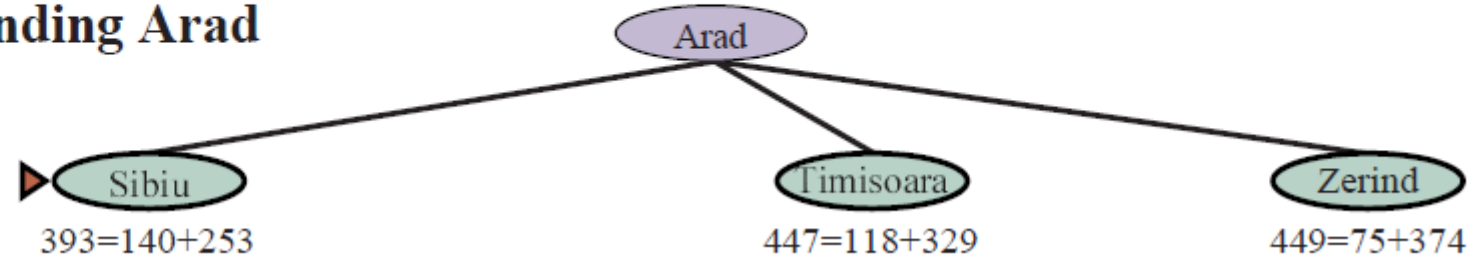
- uspořádané prohledávání s evaluační funkcí $f(n) = g(n) + h(n)$
 - $g(n)$ je cena cesty z počátečního stavu do uzlu n
 - $h(n)$ je odhadovaná cena nejkratší cesty (heuristika) z uzlu n do cílového stavu
 - $f(n)$ = odhadovaná cena nejlepší cesty z počátečního stavu přes uzel n do cíle
 - úplný algoritmus
 - cenově optimální, pokud je heuristika tzv. přípustná (tj. odhaduje nižší cenu, než je ta skutečně dosažitelná)
 - velké nároky na paměť
 - silnější vlastnost heuristiky – konzistence:
platí trojúhelníková nerovnost $h(n) \leq c(n, a, n') + h(n')$
 - konzistentní heuristika - každý stav je už při prvním objevení na optimální cestě
 - ořezávání větví, které nevedou k řešení
-

INFORMOVANÉ PROHLEDÁVÁNÍ - A*

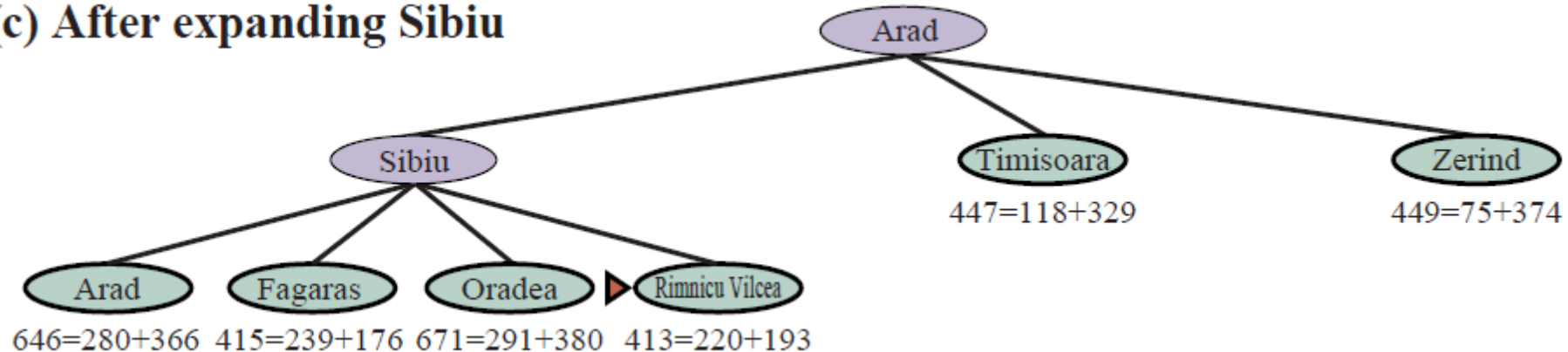
(a) The initial state



(b) After expanding Arad

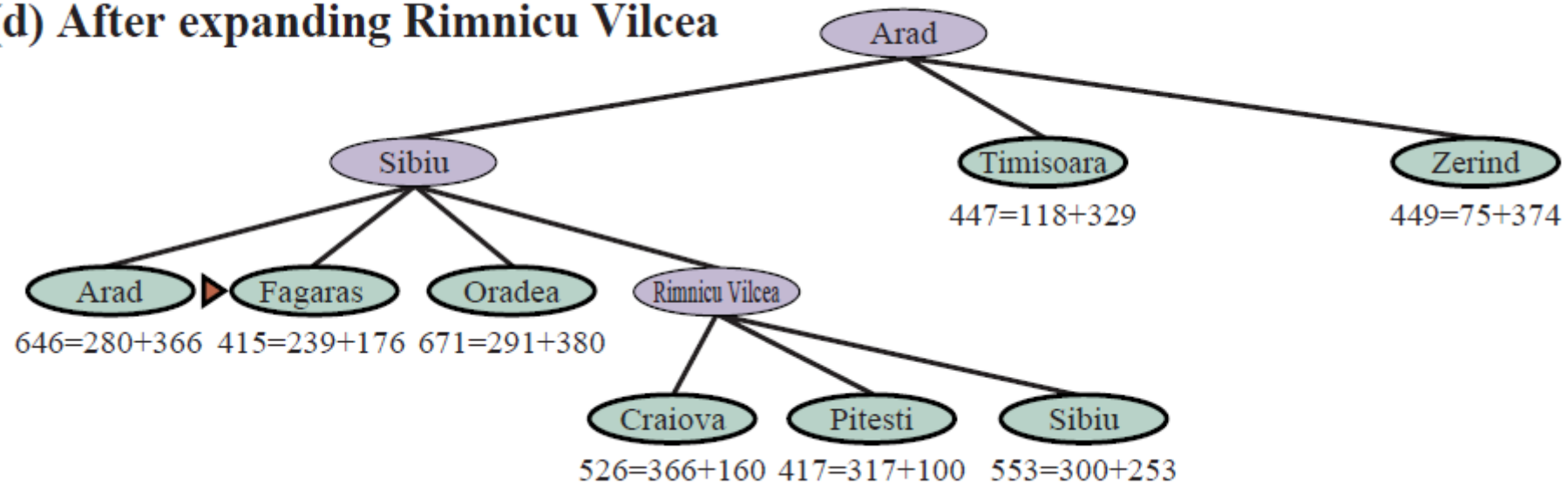


(c) After expanding Sibiu



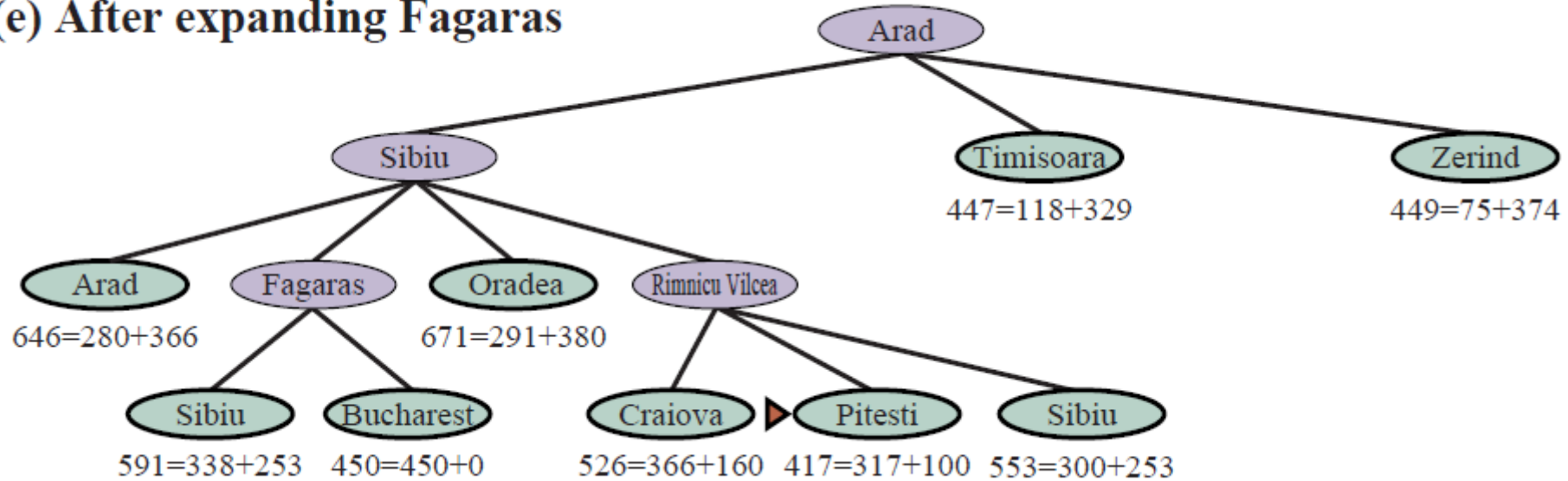
INFORMOVANÉ PROHLEDÁVÁNÍ - A*

(d) After expanding Rimnicu Vilcea



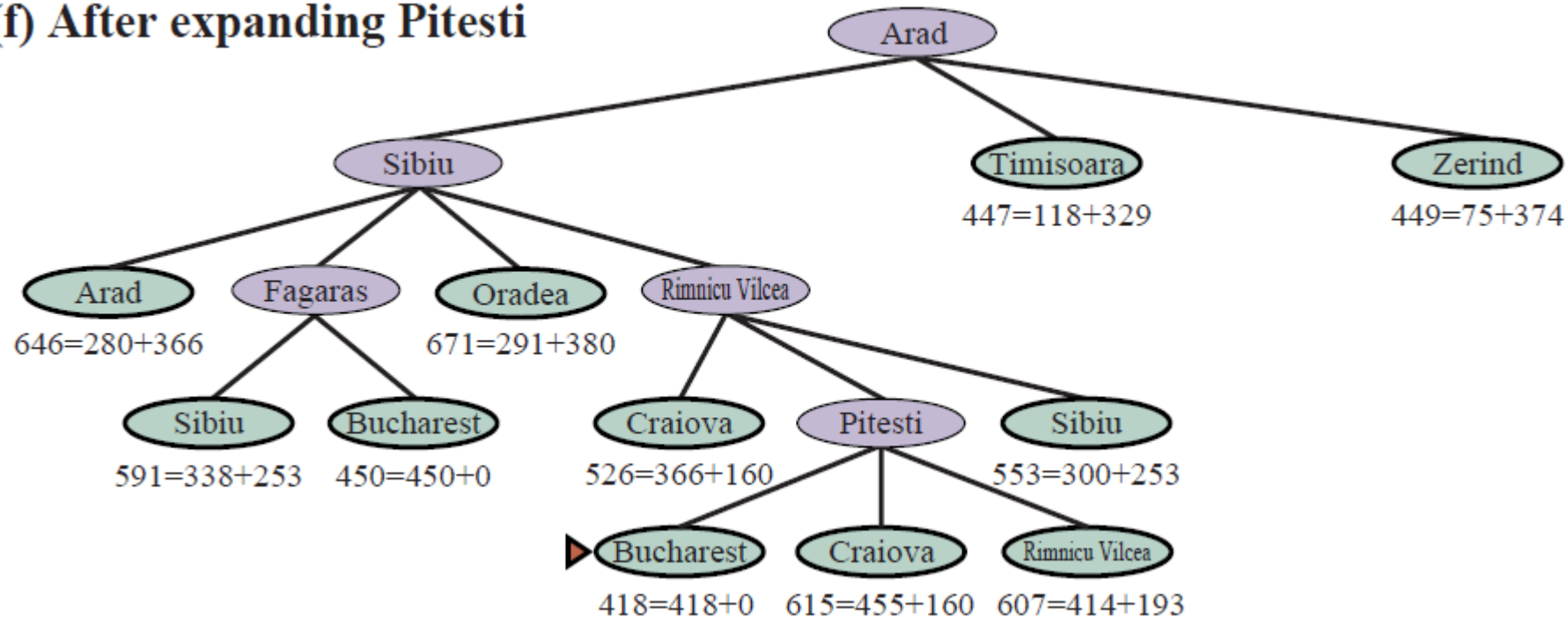
INFORMOVANÉ PROHLEDÁVÁNÍ - A*

(e) After expanding Fagaras



INFORMOVANÉ PROHLEDÁVÁNÍ - A*

(f) After expanding Pitesti



Bukurešť se poprvé objeví ve frontě v kroku (e), ale tento uzel není vybrán k rozšíření, protože $f(\text{Bukurešť}) = 450 > f(\text{Pitesti}) = 417$, tj. očekává se, že nejkratší cesta povede tudy. V kroku (f) jsou aktualizovány náklady a první cesta do Bukurešti má teď nejnížší celkové náklady, $f = 418$, a je tedy vybrána a poté i potvrzena jako optimální řešení.

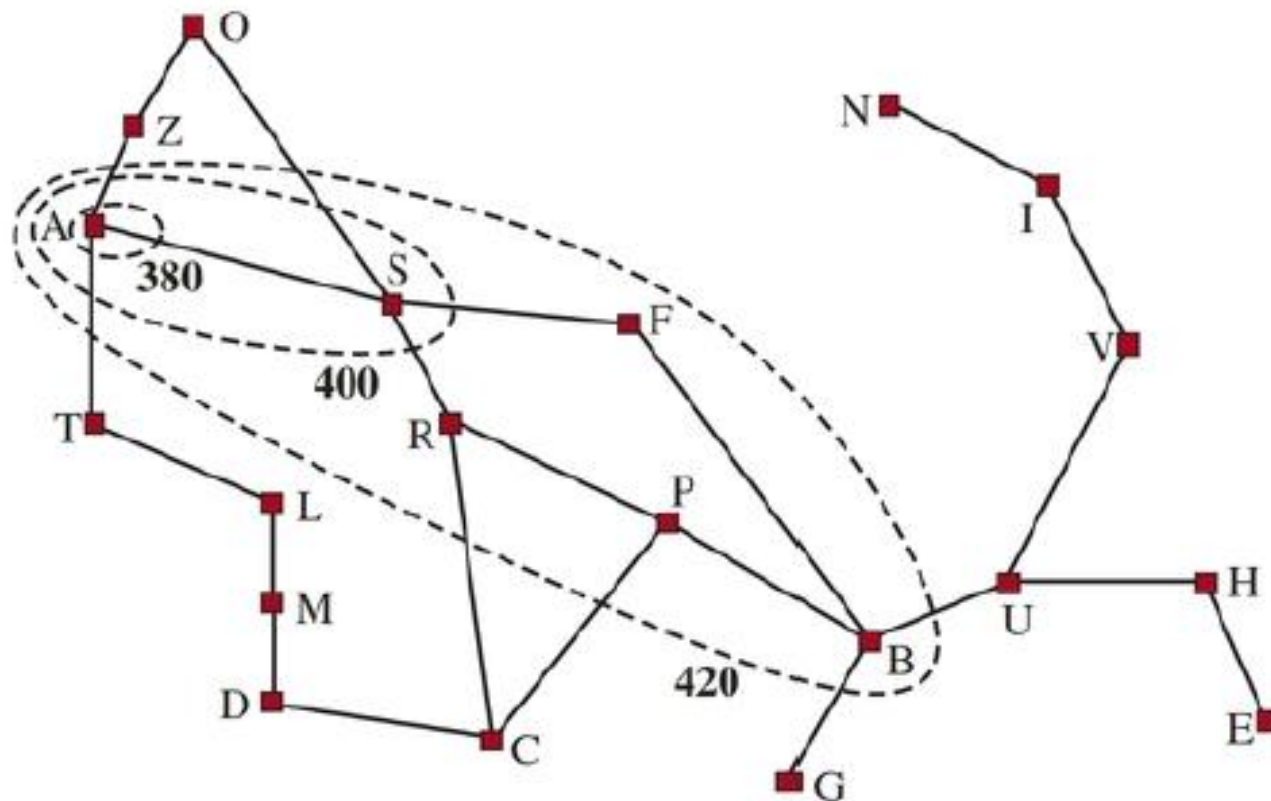
INFORMOVANÉ PROHLEDÁVÁNÍ

vrstevnice prohledávání

pro všechny uzly

uvnitř vrstevnice K

platí $f(n) = g(n) + h(n) \leq K$



INFORMOVANÉ PROHLEDÁVÁNÍ

Nepřípustné heuristiky

- A* expanduje mnoho uzlů → velký nárok na paměť
- suboptimální prohledávání – A* s nepřípustnou heuristikou – riziko nenalezení cíle
- příklad: letecká vzdálenost vynásobená „objížďkovým“ koeficientem (1.2 – 1.6)

Vážené prohledávání A*

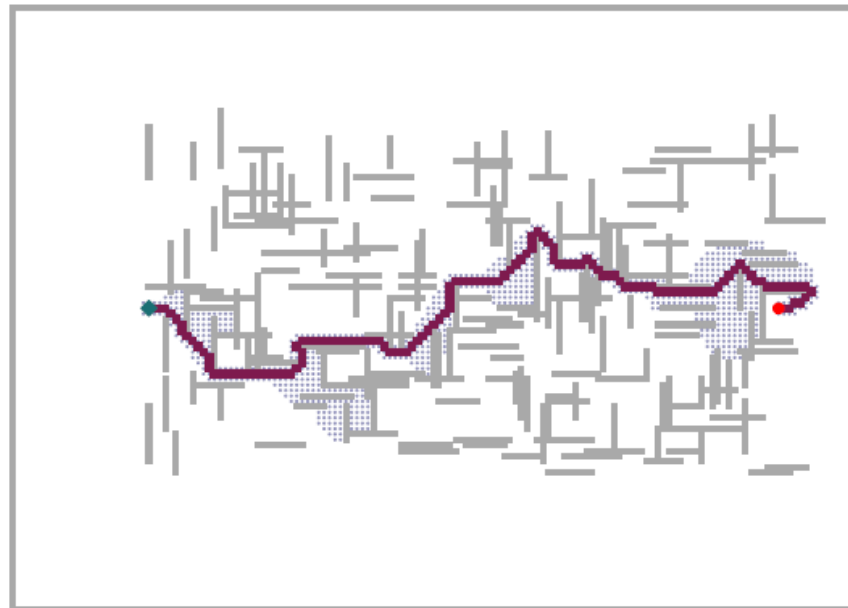
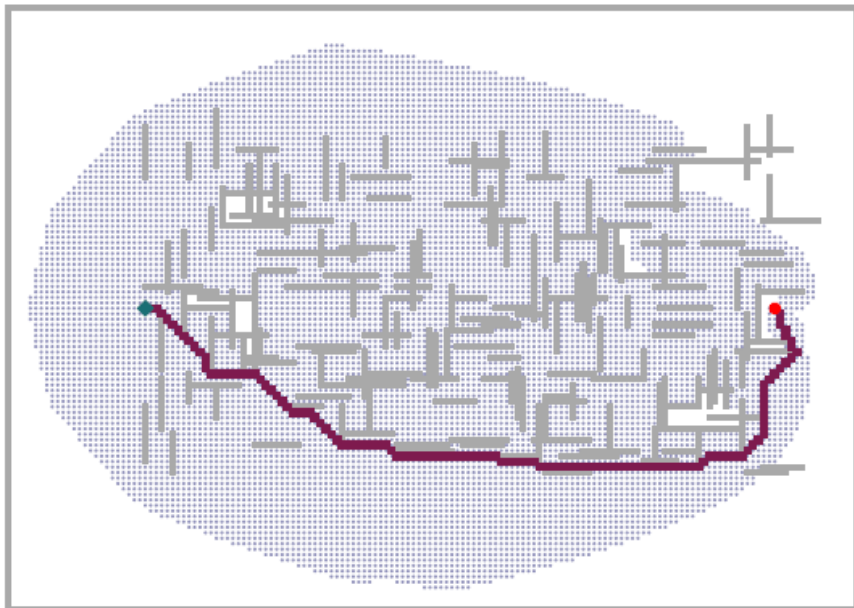
- $f(n) = g(n) + W h(n)$, $W > 1$
- rychlejší, ale dražší

A* search: $g(n) + h(n)$ ($W = 1$)

Uniform-cost search: $g(n)$ ($W = 0$)

Greedy best-first search: $h(n)$ ($W = \infty$)

Weighted A* search: $g(n) + W \times h(n)$ ($1 < W < \infty$)

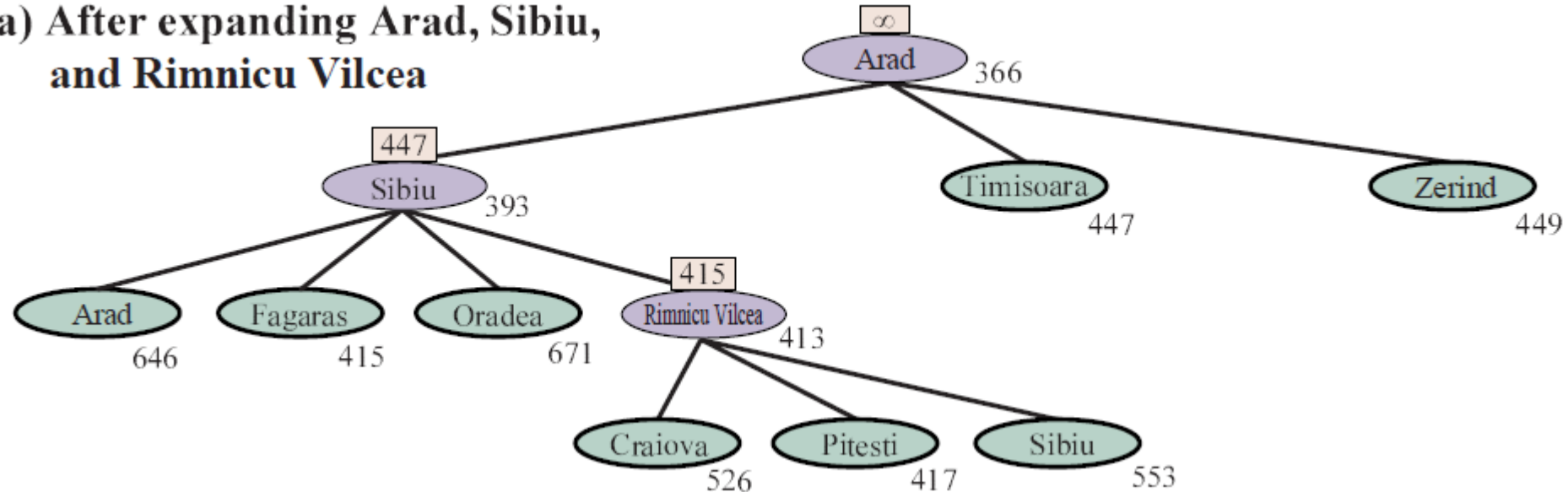


INFORMOVANÉ PROHLEDÁVÁNÍ

Prohledávání s omezenou pamětí

- odstranění stavů z paměti, když se očekává, že již nebudou potřeba
- paprskové prohledávání – omezuje velikost fronty, udržuje daný počet uzlů s nejlepší hodnotou $h(n)$, suboptimální
- IDA* - iterativně se prohlubující A* - šetří paměť za cenu možného vícenásobného procházení uzlů
- RBFS - rekurzivní uspořádané prohledávání

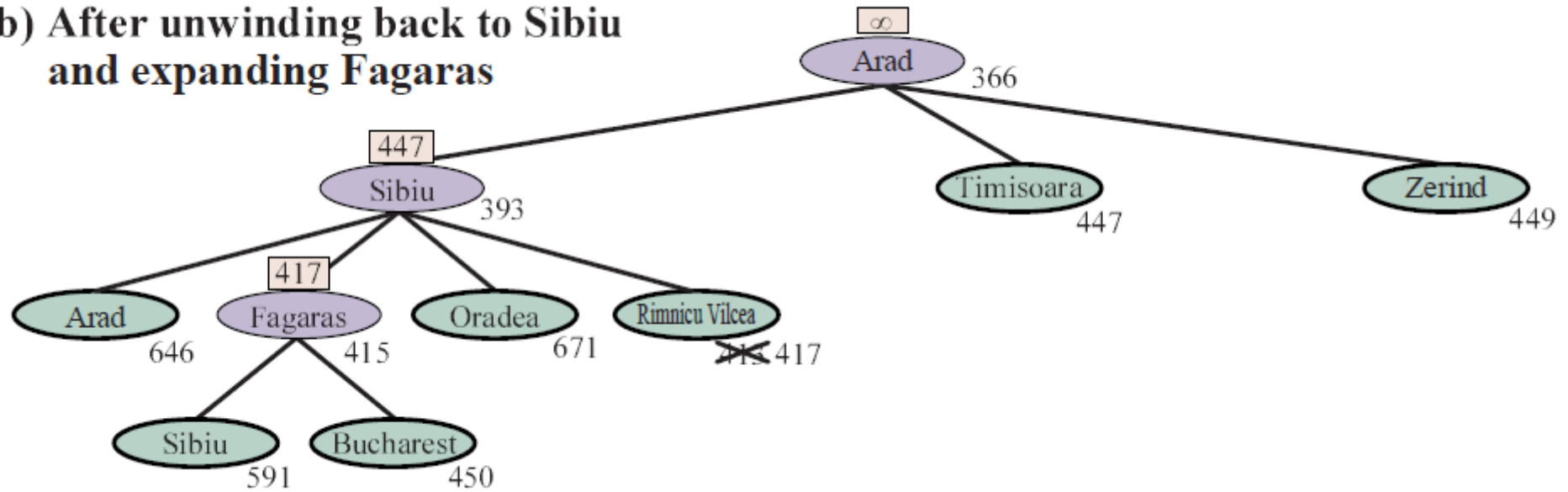
(a) After expanding Arad, Sibiu, and Rimnicu Vilcea



INFORMOVANÉ PROHLEDÁVÁNÍ

RBFS - rekurzivní uspořádané prohledávání

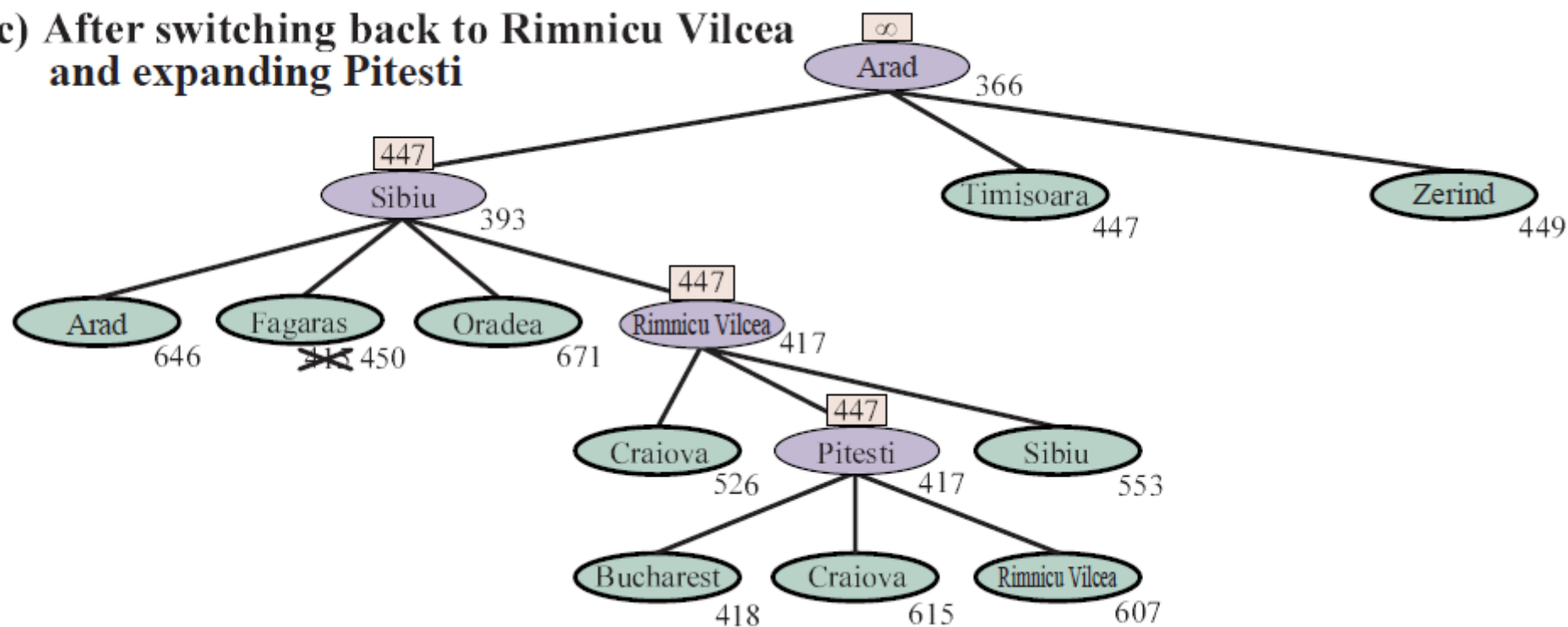
(b) After unwinding back to Sibiu
and expanding Fagaras



INFORMOVANÉ PROHLEDÁVÁNÍ

RBFS - rekurzivní uspořádané prohledávání

(c) After switching back to Rimnicu Vilcea
and expanding Pitesti

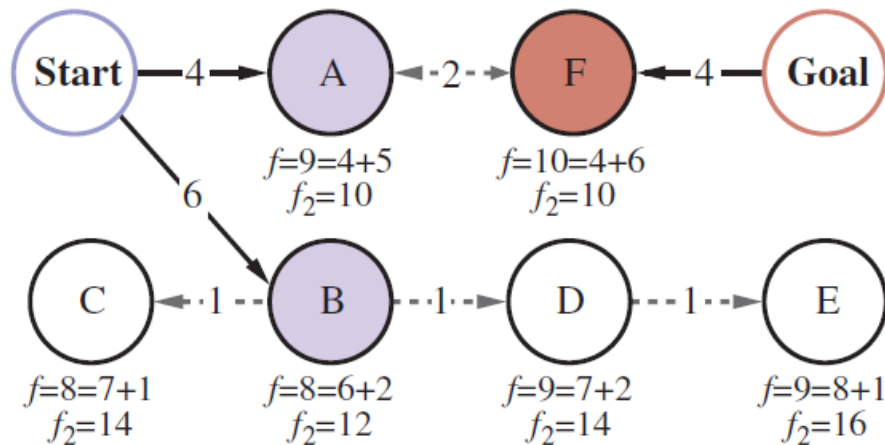


INFORMOVANÉ PROHLEDÁVÁNÍ

Obousměrné heuristické prohledávání

- použití $f(n) = g(n) + h(n)$ nezaručuje cenovou optimalitu
 - volíme $f_F(n) = g_F(n) + h_F(n)$ pro dopředný směr, $f_B(n) = g_B(n) + h_B(n)$ pro zpětný směr
 - dolní mez ceny řešení, pokud jsme došli od startu do uzlu m a z cíle do uzlu n
 $lb(m,n) = \max(g_F(m) + g_B(n), f_F(m), f_B(n))$ --> nevýhoda: nevíme, který uzel z fronty expandovat první
 - varianta $f2 = \max(2g, g + h)$ --> expandujeme uzel s nejmenší hodnotou $f2$ z libovolné fronty
-

OBOUSMĚRNÉ INFORMOVANÉ PROHLEDÁVÁNÍ



pro výběr dalšího kroku zkoumáme dvojice uzlů,
z každé fronty jeden, nikoli jednotlivé uzly

expanduje se uzel, který minimalizuje hodnotu **f2**;
uzel být z libovolné hranice

Obousměrné prohledávání udržuje dvě hranice:

uzly A a B jsou následníci Startu; uzel F inverzním následníkem Cíle.

Každý uzel je ohodnocen funkcemi **f = g + h** a **f2 = max(2g, g + h)**.

Hodnoty **g** jsou součtem ceny akcí, viz šipky; hodnoty **h** odpovídají vybrané heuristické funkci.

Optimální řešení (Start – A – F – Cíl) má náklady $C^* = 4 + 2 + 4 = 10$, to znamená, že obousměrný algoritmus by neměl expandovat žádný uzel s $g > C^*/2 = 5$; a skutečně dalším expandovaným uzlem by byl A nebo F (každý s $g = 4$), což nás vede k optimálnímu řešení.

Kdybychom nejprve expandovali uzel s nejnižšími náklady **f**, pak by na řadu přišly uzly B a C a D a E, které ale všechny mají **g > C*/2**, a proto se nikdy neexpandují, když použijeme vyhodnocovací funkci **f2**.

HEURISTICKÉ FUNKCE

- $h1$ = počet špatně umístěných dílků (bez mezery)
- $h2$ = součet vzdáleností všech dílků do cíle (Manhattanská metrika)
- efektivní faktor větvení - pokud N je celkový počet uzlů generovaných algoritmem A* a hloubka řešení je d , potom b^* je faktor větvení, jaký by měl mít pravidelný strom hloubky d , aby obsahoval $N + 1$ uzlů.

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

7	2	4
5		6
8	3	1

	1	2
3	4	5
6	7	8

HEURISTICKÉ FUNKCE

Generování heuristik z relaxovaných úloh

- relaxace = uvolnění některých omezení na akce
- graf relaxované úlohy je nadgrafem původního stavového prostoru
- odstraněním omezení vzniknou nové hrany
- cena optimálního řešení relaxované úlohy je přípustnou heuristikou pro původní úlohu

Příklad:

Původní úloha – kostička se může pohybovat z políčka X na políčko Y,
pokud X sousedí s Y a Y je prázdné

Relaxace – a) kostička se může pohybovat z políčka X na políčko Y, pokud X sousedí s Y
– b) kostička se může pohybovat z políčka X na políčko Y, pokud Y je prázdné
– c) kostička se může pohybovat z políčka X na políčko Y

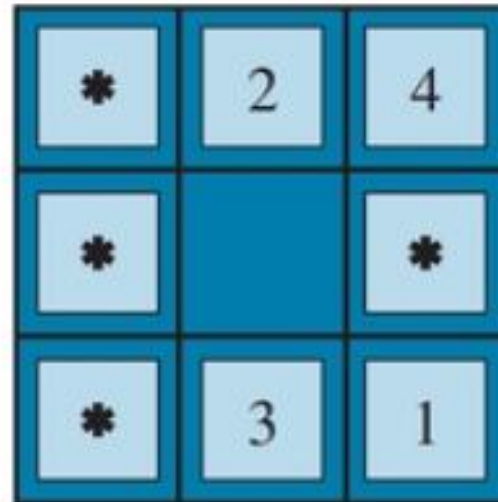
Z relaxace a) můžeme odvodit heuristiku h_2 na přechozí straně

Z relaxace c) můžeme odvodit heuristiku h_1 na přechozí straně

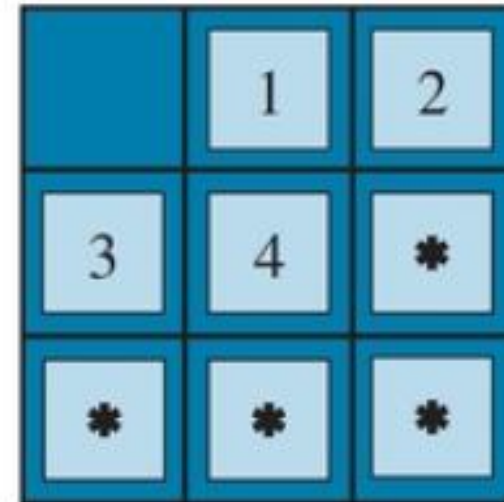
HEURISTICKÉ FUNKCE

Generování heuristik z podproblémů

- cena optimálního řešení podproblému je dolní hranicí ceny původního problému
- databáze vzorů – ukládá přesné řešení (ceny) všech možných podproblémů
 - zde všechny kombinace 4 dílků a mezery ($9 \times 8 \times 7 \times 6 \times 5 = 15.120$)
- poloha ostatních dílků neovlivní postup řešení podproblému, ale zvýší cenu řešení původního problému



Start State



Goal State

SHRNUTÍ

V této kapitole byly představeny prohledávací algoritmy, které může agent použít k výběru posloupnosti akcí v prostředích, která jsou epizodická, jedno-agentní, plně pozorovatelná, deterministická, statická, diskrétní a zcela známá.

Než může agent začít prohledávat, musí být dobře formulován řešený problém, který se skládá z pěti částí: počátečního stavu, množiny akcí, přechodového modelu popisujícího výsledky těchto akcí, množiny cílových stavů a cenu akcí.

Prostředí problému je reprezentováno grafem stavového prostoru. Cesta stavovým prostorem (posloupnost akcí) z počátečního stavu do cílového stavu se nazývá řešení. Prohledávání může být efektivnější, pokud máme k dispozici heuristickou funkci, která odhaduje, jak daleko je daný stav od cíle.

Prohledávací algoritmy obecně uvažují atomické stavy a akce, tj. bez jakékoliv vnitřní struktury.

Prohledávací algoritmy se posuzují na základě úplnosti, nákladové optimality, časové složitosti a prostorové složitosti. Při návrhu je třeba hledat kompromis mezi časem, který zabere prohledávání, množstvím dostupné paměti a kvalitou řešení.

Zdroj:

Russell, S. J., Norvig, P. *Artificial Intelligence: A Modern Approach*. 4. vyd. Pearson, 2021
